Head of Claude Code: What Happens After Coding Is Solved

---

1. Source Info

Title: Head of Claude Code: What happens after coding is solved
Speaker: Boris Cherny (Creator & Head of Claude Code, Anthropic)
Host: Lenny Rachitsky (Lenny's Podcast)
Date: 19 February 2026
Views: 423,000+ at time of review
Link: https://www.youtube.com/watch?v=We7BZVKbCVw

---

2. TL;DR

Boris Cherny, creator of Claude Code at Anthropic, makes the case that coding (the act of writing software by hand) is now "largely solved" by AI. He has not written a single line of code by hand since November 2025, yet he ships 10 to 30 pull requests per day by running multiple AI agents simultaneously. The conversation covers how Claude Code grew from a weekend prototype to powering 4% of all public GitHub commits, the counterintuitive product principles that drove its success, why small underfunded teams build better AI products, the printing press as a historical analogy for what is happening to software, and what comes next for engineers, product managers, and designers in a world where implementation is no longer the bottleneck.

---

3. Core Thesis

Boris Cherny's central argument is that coding, defined specifically as the mechanical act of translating human intent into working software, is now a solved problem. He is careful with this claim. He does not mean that software engineering is solved, or that building great products is solved, or that the need for human judgment has disappeared. What he means is that the implementation layer (writing functions, debugging errors, handling boilerplate, running tests, iterating on pull requests) can now be reliably delegated to AI.

He offers himself as the primary evidence. Since November 2025, 100% of his code has been authored by Claude Code. He runs five or more AI agents simultaneously, each working on independent tasks across separate terminal windows. One agent runs a test suite while another refactors a module while a third drafts documentation. He monitors their progress through iTerm2 system notifications and intervenes only when an agent gets stuck or needs directional input. The result is that he ships more code in a day than most engineers ship in a week, and his role has shifted from writing code to directing intent.

The deeper layer of his thesis concerns what happens after implementation becomes cheap. Boris draws a parallel to the printing press: within 50 years of Gutenberg's invention, more printed material existed than in the previous thousand years. Costs dropped by a factor of 100. But literacy took 200 years to reach 70% of the global population, because it required infrastructure, education systems, and cultural change. Boris believes AI is doing the same thing to programming. The cost of producing software has already dropped dramatically. But the full democratization, where anyone can describe what they want in natural language and get working software, will take years to fully unfold. For practitioners right now, the implication is stark: the value of knowing how to code is declining rapidly, while the value of knowing what to build and why is increasing just as fast.

---

4. Key Insights & Frameworks

4.1 The Origin Story of Claude Code

Claude Code started as a weekend hack. Boris describes how he built the initial prototype

quickly, almost as an experiment, and the team was surprised by how intensely people responded to it. What began as a simple terminal-based tool for interacting with Claude while coding turned into one of Anthropic's most important products. The growth trajectory was steep: Claude Code went from a quick internal prototype to powering 4% of all public GitHub commits, with daily active users doubling in a single month.

What makes this origin story significant from a product perspective is that Boris didn't set out to build a massive platform. He built something he personally wanted to use, shipped it, and discovered that the demand was enormous. He describes this using the concept of latent demand: millions of engineers wanted a better way to work with AI on real code, and millions of non-engineers wanted to build software but couldn't because the barrier was too high. Claude Code didn't create this demand; it tapped into demand that was already there, waiting for the right tool to unlock it.

Boris compares this to drilling for water. The demand was underground. The product just had to reach it. For growth practitioners, this is an important framing: the most successful products in the AI era may not be ones that manufacture new behaviors, but ones that reveal and serve latent needs that were previously too expensive or difficult to address.

4.2 Why Boris Left for Cursor and Came Back in Two Weeks

One of the more revealing segments of the conversation is Boris's admission that he briefly left Anthropic for Cursor, one of the leading AI-powered code editors, and came back after just two weeks. Lenny probes this, and Boris explains that the experience actually strengthened his conviction in Anthropic's approach.

At Cursor, he got firsthand exposure to how a competitor was thinking about AI-assisted coding. But what he realized was that the underlying model quality mattered far more than the editor's user interface or feature set. He came back to Anthropic because he believed Claude's model (specifically Opus) was fundamentally better at coding tasks. He describes it this way: even though Opus is bigger and slower than Sonnet, you have to steer it less and it's better at tool use, so it ends up being faster in practice because you spend less time correcting it.

This is a significant insight for anyone evaluating AI tools. The market tends to focus on surface-level features (UI polish, keyboard shortcuts, integrations), but Boris's experience

suggests that the quality of the underlying model is the variable that actually determines productivity. The best coding tool is whichever one has the best model, full stop.

4.3 The "Coding Is Solved" Claim, With Its Qualifications

Boris's statement that "coding is largely solved" is the headline claim of the conversation, and it's worth unpacking exactly what he means and what he does not mean.

He is specific: the mechanical act of writing code (typing syntax, implementing functions, writing boilerplate, fixing bugs, writing tests) is what's solved. AI agents can now do this reliably for the vast majority of tasks. What is NOT solved is software engineering in the broader sense: deciding what to build, understanding user needs, making architectural tradeoffs, designing systems that scale, and exercising judgment about what matters. Boris sees these higher-order skills as becoming more valuable, not less, precisely because the implementation layer is now cheap.

He also acknowledges that AI coding is not perfect. Agents still get stuck. They still produce code that needs review. They sometimes go in the wrong direction and need to be redirected. Boris describes his own experience of anxiety when agents aren't working well, when they're spinning on a problem or producing low-quality output. He doesn't pretend this is a friction-free experience. But his argument is that even with these imperfections, the productivity gain is so large that it fundamentally changes how software gets built.

There's an important nuance here for growth teams: "solved" does not mean "fire your engineers." It means the bottleneck shifts. The constraint is no longer "can we build this?" but "should we build this?" and "what should we build next?" For growth engineering specifically, this means the limiting factor on experimentation is no longer implementation capacity. It's experiment design, metric selection, and strategic prioritization.

4.4 Boris's Current Coding Workflow in Detail

Boris walks through his actual day-to-day workflow in considerable detail, and this is one of the most practically valuable segments of the conversation.

He uses Claude Code exclusively in the terminal (not in an IDE). He runs five or more agent

sessions simultaneously across different terminal windows in iTerm2. Each agent is assigned a discrete, well-scoped task: a feature branch, a bug fix, a refactor, a documentation update, or a test suite. He monitors progress through iTerm2's system notifications, which alert him when an agent completes a task or hits a blocker.

His process for a typical task is: describe what he wants in natural language, let the agent work, review the output, and either approve it or give the agent additional direction. He describes shipping 10, 20, sometimes 30 pull requests in a single day, each one written entirely by Claude Code. He says he hasn't edited a single line of code by hand since November.

On top of this, he runs 5 to 10 Claude sessions in his browser on claude.ai for non-coding tasks: drafting documents, analyzing data, brainstorming ideas. His entire workday is mediated through AI agents working in parallel.

The key lesson for practitioners is that this workflow requires a different skill set than traditional coding. Instead of writing code, Boris spends his time on task decomposition (breaking large problems into agent-sized chunks), quality review (evaluating agent output for correctness and quality), and strategic direction (deciding what to build and in what order). These are fundamentally different competencies than syntax and debugging.

4.5 The Printing Press Analogy

This is the most fully developed framework in the conversation, and Boris presents it with historical specificity.

Before the printing press, Europe's literacy rate was under 1%. Within 50 years of Gutenberg's invention, more printed material was produced than in the previous thousand years. The cost of producing written material dropped by a factor of 100 over the next 50 years. But here's the critical nuance: literacy itself took roughly 200 years to reach 70% globally. The technology arrived fast, but the societal transformation was slow, because it required education systems, free time (people had to not be working on farms all day), and cultural shifts in how knowledge was valued and transmitted.

Boris then surfaces a fascinating historical detail that makes this more than a standard analogy. He references a document from the 1400s in which a scribe was interviewed about

how they felt about the printing press. Contrary to the standard narrative of displaced workers resisting new technology, the scribe was actually excited. They saw the press as liberating them from the tedious work of copying manuscripts by hand, freeing them to focus on the higher-value work of composition and original writing.

Boris explicitly identifies with this scribe. He says he has never enjoyed coding as much as he does today, precisely because he doesn't have to deal with the minutiae anymore. The mundane parts of coding (syntax, boilerplate, debugging semicolons) are gone. What remains is the creative and strategic work: architecture, product thinking, and directing AI agents toward the right outcomes.

For practitioners, the printing press analogy offers a useful time horizon. The technology revolution is happening now, but the full societal adaptation will take years, not months. People who invest in learning to work with AI tools today are positioning themselves for a transition that will unfold over a decade or more.

4.6 The Underfunding Paradox and Team Principles

Boris makes a counterintuitive argument about team structure that deserves careful attention. He argues that the best AI products come from small, deliberately underfunded teams. The Claude Code team was kept intentionally small, and Boris believes this was a key driver of its success.

His reasoning: when you give a large team generous headcount, they spend their time building processes, hiring managers, creating review cycles, and coordinating between people. When you give a small team unlimited AI tokens instead of unlimited headcount, they spend their time building product. The constraints force them to lean on AI tools for everything, which makes them better dogfooders of their own product, which directly improves the product itself. It's a virtuous cycle that only works when the team is small enough to feel the constraints.

He extends this into a specific recommendation: organizations should give every engineer unlimited access to AI coding tokens. The cost is trivial compared to engineering salaries. At Anthropic, this policy contributed to a 200% increase in per-engineer productivity.

Boris also shares three principles he tells every new person joining the Claude Code team.

First, use the product obsessively. If you're building an AI coding tool, 100% of your own coding should be done through that tool. There is no substitute for dogfooding. Second, ship fast, iterate faster. The Claude Code team ships multiple times per day. Speed is treated as a feature, not a risk. Third, stay uncomfortable. If you're not anxious about how fast things are changing, you're not paying close enough attention. The discomfort is a signal that you're working at the frontier.

4.7 Which Roles Will AI Transform Next

Boris spends a meaningful portion of the conversation discussing which professional roles will be affected by AI and in what order. He is direct about his predictions.

He believes the title "software engineer" will start to go away, replaced by "builder." This isn't just semantic; it reflects a real shift in what the job entails. A builder is someone who creates things using whatever tools are available, rather than someone who writes code in a specific language. The identity shift matters because it changes what people optimize for: outcomes rather than technical process.

He extends this to product managers and designers. On the Claude Code team, everyone codes, including the product manager, the engineering manager, the designer, the finance person, and the data scientist. AI makes this possible because you don't need deep expertise in a domain to direct an AI agent effectively; you need enough understanding to evaluate the output and give useful direction.

Boris also discusses a poll Lenny references about which roles are most enjoying working with AI. The data suggests that people who are embracing AI tools and crossing disciplinary boundaries are finding their work more rewarding, while those who are clinging to narrow specializations are feeling more threatened. Boris's advice is clear: be a generalist. The era of the narrow specialist is ending.

4.8 How Cowork Was Built in 10 Days

Boris describes Cowork (Anthropic's AI assistant that can operate a web browser autonomously) and reveals that it was built in just 10 days. This anecdote serves as a concrete illustration of several themes in the conversation.

First, it demonstrates the speed that's possible when a small team uses AI tools intensively. Ten days from concept to a working product that can open Chrome, navigate web pages, fill out forms, and complete multi-step workflows autonomously. Boris mentions an example of an Anthropic employee who went on parental leave and used Cowork to fill out medical forms automatically: it loaded the browser, logged in, filled out the PDFs, and submitted them.

Second, it illustrates the latent demand principle again. The team tried a similar experiment a year earlier and it didn't work because the model wasn't ready. But now the model quality had crossed a threshold where browser automation actually worked reliably. The demand was there all along; the technology just had to catch up.

4.9 The Three Layers of AI Safety at Anthropic

Boris discusses Anthropic's approach to AI safety with genuine depth, which is notable given that safety is sometimes treated as an afterthought in product-focused conversations. He describes three distinct layers.

The first layer is model-level safety: the training and alignment work that ensures the base model behaves responsibly. The second layer is product-level safety: the guardrails, filters, and controls built into products like Claude Code that prevent misuse. The third layer is organizational-level safety: the culture, processes, and governance structures at Anthropic that ensure safety considerations are integrated into every decision.

Boris frames this as non-negotiable. He doesn't present safety as a tradeoff with capability or speed; he presents it as a prerequisite for building products that people can trust and that companies can deploy at scale.

4.10 Anxiety at the Frontier

One of the most human moments in the conversation is Boris's candid discussion of anxiety. He doesn't project the confident calm that most tech leaders default to when discussing AI. Instead, he acknowledges that working at the frontier of AI is genuinely anxiety-inducing.

He describes the specific experience of anxiety when AI agents aren't working well: when

they're spinning, producing bad output, or going in the wrong direction. He also describes a broader existential anxiety about the pace of change and the implications for people's careers and livelihoods.

His advice on this is striking: if you're not anxious, you're not paying attention. He normalizes the discomfort rather than dismissing it. And he suggests that the anxiety itself can be productive if channeled into action: experimenting with tools, learning new skills, and staying on the frontier rather than retreating from it.

4.11 Boris's Ukrainian Roots and Post-AGI Plans

Toward the end of the conversation, Boris discusses his Ukrainian background and how it has shaped his perspective. He connects the resilience and resourcefulness required by his upbringing to the mindset needed to work in a field that is changing as rapidly as AI.

When asked about his post-AGI plans (what he would do if artificial general intelligence were achieved), Boris says he's interested in education. This is consistent with the printing press analogy: if AI is democratizing the ability to build software, the bottleneck becomes education, helping people understand what's possible and how to direct these tools effectively. It's a long-term, systems-level view that goes beyond the immediate product work.

4.12 Advice for Building AI Products

In the final sections of the conversation, Boris offers extended tactical advice for anyone building AI products. He recommends using the best available model, even if it's more expensive or slower, because the reduction in steering and correction time more than compensates. He emphasizes the importance of experimentation: try things, ship them, see what works, and iterate quickly. He warns against over-planning and under-shipping.

He also discusses his thoughts on OpenAI's Codex, acknowledging it as a competitor but expressing confidence that model quality will remain the decisive factor, and that Anthropic's models are ahead on the dimensions that matter most for coding: tool use, instruction following, and the ability to work autonomously on complex multi-step tasks.

5. Tactical Playbook

5.1 Set Up a Multi-Agent Workflow

Boris runs five or more Claude Code agents simultaneously, each on an independent task. He monitors them through terminal notifications and intervenes only when needed.

What to do: Set up your terminal to support parallel agent sessions. Start with two agents on independent tasks and scale up as you build comfort with multi-stream supervision. The skill you're developing is task decomposition: breaking large problems into agent-sized chunks that can run in parallel.

5.2 Give Your Team Unlimited AI Tokens

At Anthropic, giving engineers unlimited tokens contributed to a 200% increase in per-engineer productivity. The cost of tokens is trivial compared to engineering compensation.

What to do: Calculate the annual cost of unlimited AI tokens for your growth engineering team. Compare it to the cost of one additional engineer. Present this to leadership as a headcount-equivalent investment that ships immediately with zero ramp time.

5.3 Redefine Your Experiment Backlog

When implementation cost drops by 5 to 10x, experiments that were previously too expensive to justify become viable. The constraint shifts from "can we build this?" to "should we build this?"

What to do: Review your current experiment backlog. Identify every experiment that was deprioritized because of implementation complexity. Reassess each one under the assumption

that implementation cost is 10x lower than when it was originally estimated. Reprioritize based on expected impact, not expected effort.

5.4 Dogfood Relentlessly

Boris's first principle for new team members is to use the product obsessively. For growth teams, this means using your own product the way a new user would, regularly and critically.

What to do: Schedule a weekly 30-minute session where the entire growth team uses the product from a new-user perspective. Document friction points, confusing flows, and moments of delight. Feed these directly into your experiment pipeline.

5.5 Frame Work as Outcomes, Not Implementation

Boris predicts the "software engineer" title will evolve into "builder." The identity shift matters because it changes what people optimize for.

What to do: In your next sprint planning, frame every ticket in terms of the outcome it produces ("increase activation rate by 5%") rather than the implementation it requires ("add tooltip to onboarding step 3"). This aligns the team's attention with value creation rather than task completion.

5.6 Use AI on Real Production Work, Not Toy Projects

Boris is emphatic that the only way to build genuine intuition for AI-assisted development is to use it on real work. The gap between demo-quality output and production-quality output is where all the real learning happens.

What to do: Pick one real production task this week and commit to completing it entirely through AI-assisted coding. Document what worked, what failed, and where you had to intervene. Share the learnings with your team.

5.7 Invest in Review Infrastructure Before Scaling Velocity

The research data shows that teams with high AI adoption merge 98% more pull requests, but PR review time increases 91%. If you scale code generation without scaling review capacity, you'll create a bottleneck.

What to do: Before pushing your team to generate more code with AI, invest in automated testing, CI/CD guardrails, linting, and observability. These are the systems that let you safely absorb higher velocity without sacrificing quality.

---

6. Growth Engineering Lens

6.1 What "Coding Is Solved" Actually Means for Experiment Velocity

Boris ships 10 to 30 PRs per day using multiple AI agents in parallel. For a growth engineer, this claim lands differently than it does for a product engineer, because growth engineering is fundamentally about shipping to learn, not shipping to build. The "tent vs. skyscraper" philosophy (coined by Gergely Orosz in his Pragmatic Engineer deep-dive on growth engineering) captures this perfectly: growth engineers deliberately take shortcuts inappropriate for long-term systems, use lightweight approaches, and accept reduced code coverage when compensated by robust monitoring. If AI makes tents even cheaper and faster to erect, the entire experimentation economics shift.

Consider the benchmarks. A typical growth engineering team targets 4 to 6 experiments per month. Google's top-performing product teams achieve 50 or more experiments annually. At the extreme end, Airbnb increased from 100 to 700 experiments per week over two years and now runs 500+ tests concurrently. LinkedIn runs 35,000 concurrent experiments. Boris's workflow, where a single engineer ships 10 to 30 PRs per day, suggests that AI-assisted growth engineers could push individual experiment throughput toward what previously required entire teams. The constraint shifts from "how fast can we code this variant?" to "how fast can we design a valid hypothesis and instrument the right metrics?"

But there is a critical nuance the research surfaces: impact per test peaks at 1 to 10 annual tests per engineer, and beyond 30 tests per engineer, expected impact drops by 87%. Running more experiments is not inherently better. The discipline is knowing which experiments to run, not just shipping more of them. Boris's emphasis on directing intent rather than writing code maps directly onto this: the growth engineer's value increasingly lives in hypothesis quality, metric selection, and strategic prioritization, not implementation speed.

6.2 The Underfunding Paradox Applied to Growth Teams

Boris's counterintuitive claim that small, deliberately underfunded teams build better AI products has a direct analog in growth engineering. Elena Verna's research on growth team failures highlights that the most common failure mode is growth teams operating too slowly like core engineering: overdeveloping projects, getting caught up in dependencies, building skyscrapers when they need tents. Boris's recommendation (give teams unlimited AI tokens instead of unlimited headcount) is essentially the growth engineering philosophy taken to its logical conclusion.

At Anthropic, unlimited tokens contributed to a 200% increase in per-engineer productivity. For a growth engineering team, this translates to a concrete budget argument. One senior growth engineer costs $120,000 to $475,000 per year (Glassdoor 2025 data for the range from 25th to 75th percentile at top companies). Unlimited AI coding tokens for an existing team might cost $500 to $2,000 per engineer per month. The math is straightforward: you can buy the equivalent of additional engineering capacity at 5 to 10% of the cost of a new hire, with zero ramp time, zero management overhead, and no recruiting pipeline (which itself takes 58% of employers longer than usual in 2024, with one-third of the best applicants lost to slow processes).

This argument is strongest for growth teams specifically because growth engineering work is high-velocity, experiment-heavy, and crosses many codebases. GitHub Copilot data from 2025 shows it generates 46% of code written by developers and reduces PR cycle time from 9.6 days to 2.4 days (a 75% reduction). Growth engineers, who routinely touch frontend onboarding flows, backend API endpoints, analytics pipelines, and feature flag configurations in a single sprint, benefit disproportionately from this kind of cross-codebase acceleration.

6.3 Experimentation Infrastructure in the AI Era

Boris's claim that coding is solved arrives at a moment when the experimentation platform landscape is undergoing its biggest consolidation in years. In 2025, OpenAI acquired Statsig for $1.1 billion, and Datadog acquired Eppo for $220 million. These acquisitions signal something important: experimentation infrastructure is now considered critical to AI product development. As Sequoia noted during the Statsig deal, "AI can create infinite variations, but the harder problem is figuring out which one works."

For growth engineers, this consolidation changes the build-versus-buy calculus. Statsig offers sequential testing, stratified sampling, CUPED variance reduction, switchback experiments, and instant flag-to-test conversion. Eppo (now "Eppo by Datadog") brings warehouse-native architecture that integrates directly with Snowflake and Redshift, plus statistical canary testing automated based on errors, infrastructure metrics, and product telemetry. These are capabilities that previously required months of internal platform engineering.

The Alexey Test (Alexey Komissarouk's 11-step framework for evaluating growth engineering maturity) provides a useful checklist for how AI changes each step:

1. **Proper A/B testing infrastructure** : AI agents can now scaffold experiment configurations in Statsig or GrowthBook in minutes, including variant definitions, metric bindings, and segment targeting. The setup work that used to take a day becomes a prompt.
2. **Experimentation-ready codebase** : AI can generate the boilerplate (getExperimentVariant() helpers, Experiment component wrappers, experiment directories) that makes a codebase experiment-friendly. Boris's multi-agent workflow means a growth engineer could instrument an entire new surface area for experimentation in a single afternoon.
3. **Fast deployment cycles** : AI doesn't directly accelerate deployment pipelines, but it dramatically accelerates the code-to-PR portion of the cycle, meaning growth engineers spend more of their time in the deploy-measure-learn phase rather than the write-code phase.
4. **Robust quality tooling** : This is where Boris's workflow creates risk. AI-assisted code can increase issue counts approximately 1.7x if not paired with governance. For growth teams, buggy experiments pollute data, create negative user experiences, and erode trust in the experimentation platform. Error boundaries preventing cascading failures from experiment crashes, automated test suites, and on-call monitoring become more important, not less, when code generation is cheap.
5. **Engineer-generated experiment ideas** : Boris's point about the shift from implementation

to intent connects directly to step 8 of the Alexey Test, which calls for engineers to be "mini-PMs for their projects," owning experiments from conception through analysis. When coding is solved, this shift is no longer aspirational; it is the job description.

6.4 Activation and Onboarding: Where AI-Assisted Growth Engineering Hits Hardest

Growth engineering's highest-impact work often lives in activation and onboarding flows, and this is precisely where Boris's "coding is solved" claim creates the most immediate leverage. Recent practitioner data shows the magnitude of what well-executed activation experiments can achieve:

- A 12-second loom-style video placed above a form increased activation from 28% to 46% in two weeks.
- Replacing a six-step product tour with a single "run this CLI copy-paste" command achieved a 33% activation lift.
- Adding a dashboard template to onboarding flow measurably boosted activation.

Each of these experiments is relatively simple to implement: frontend changes, conditional logic, minor API integrations. They are the definition of "tent" work. With AI agents handling implementation, a growth engineer could ideate, build, and ship one of these experiments in a single day rather than a single sprint. The weekly cadence that many growth teams follow (Monday pull metrics, Tuesday flag anomalies, Wednesday ideate experiments, Thursday launch, Friday share learnings) could compress into a daily cadence for certain experiment types.

The MasterClass fake door test is another instructive example. The growth team wanted to test multi-tier pricing but faced months of complex backend work. Instead, they created a fake door offering unappealing pricing tiers during checkout, then "upgrading" customers to better rates. This required only days of frontend work while gathering critical revenue data. With AI agents, the implementation portion of even the "full" version shrinks dramatically, which means the strategic decision about whether to fake-door or build the real thing shifts: if the full implementation is nearly as fast as the fake door, the tradeoff calculus changes.

6.5 Growth Loops, Not Funnels: Engineering Compounding Systems

Boris's printing press analogy (costs drop 100x, but adoption takes 200 years) maps onto how growth engineers should think about loops versus funnels. Funnels are linear: they have a defined start and end point, and they run out of fuel. Growth loops compound because output feeds back into input. The printing press didn't just produce more books; it created a loop where more books led to higher literacy, which led to more demand for books, which drove more printing.

For growth engineers, the AI-era equivalent is this: AI makes it cheap to build the initial version of a growth loop (referral engine, viral sharing mechanic, content-generation pipeline, data flywheel). The hard part is not the initial build; it is instrumenting the loop correctly, measuring the compounding rate, and iterating on the loop mechanics to improve the viral coefficient or content output ratio. Boris's multi-agent workflow is well suited to this: one agent builds the referral flow, another instruments the analytics, a third writes the A/B test that measures the loop's coefficient. The growth engineer's job is to orchestrate these agents toward a compounding system, not to write any of the individual components.

Atlassian's growth team provides a concrete example. Their Growth engineers built an experimental feature linking Confluence spaces to Jira projects through A/B testing. After positive results, they enrolled every Jira customer in final experiments, eventually rolling out to 100% of users with measurable Confluence engagement increases. This cross-product growth loop (Jira usage driving Confluence adoption) is exactly the kind of system-level engineering where AI acceleration on the implementation side frees the growth engineer to focus on the loop design and measurement.

6.6 The Self-Serve Tooling Imperative

One of the highest-leverage moves a growth engineer can make is removing themselves as a bottleneck so colleagues from marketing, product, and data can iterate independently. This is what the Pragmatic Engineer calls "empowerment work": building tools or integrating MarTech so marketing teams can run campaigns, modify landing pages, and launch experiments without filing engineering tickets.

Boris's "coding is solved" claim accelerates this in two ways. First, growth engineers can use AI agents to build internal dashboards, no-code experiment launchers, and self-serve configuration tools much faster than before. Second (and more subtly), non-engineers on the growth team can increasingly use AI coding tools themselves to make the changes they

need. Boris describes how everyone on the Claude Code team codes, including the PM, the designer, the finance person, and the data scientist. If this pattern spreads, the traditional handoff between marketing and engineering dissolves. The growth marketer who wants to test a new CTA doesn't file a ticket; they describe the change to an AI agent and ship it themselves, with the growth engineer's role shifting to building the guardrails, review processes, and monitoring that make this safe.

6.7 The PR Review and Code Quality Bottleneck

Teams with high AI adoption merge 98% more pull requests, but PR review time increases 91%. This is the growth engineering version of Boris's "coding is solved" claim meeting reality. When code generation is cheap, human review becomes the bottleneck.

For growth teams specifically, this bottleneck is dangerous because experiment quality depends on code quality. Buggy experiment code does not just create user-facing bugs; it corrupts experiment data, invalidates statistical results, and can lead to incorrect business decisions. The Alexey Test's step 6 (sound experimental hygiene: no peeking at results prematurely, holdouts, reruns, winner's curse controls) assumes that the experiment code itself is correct. AI-generated code that subtly misconfigures a feature flag or miscounts events can silently undermine an entire quarter's worth of experimentation.

The tactical response is to invest in automated quality gates before scaling velocity. This means automated test suites with error boundaries preventing cascading failures from experiment crashes, linting rules specific to experiment code (checking for proper flag cleanup, metric binding validation, segment configuration), and monitoring that alerts on anomalous experiment data patterns (sudden enrollment drops, metric value distribution changes, impossible conversion rates). Growth engineers should treat these quality systems as the foundation that makes AI-accelerated experimentation safe, not as nice-to-haves that can be added later.

The information leakage dimension is also worth noting. Companies routinely expose experiment configurations via API routes: Lyft serves 180KB of JSON with 1,449 client variables, and Pinterest has 823 active experiments visible via API. When AI makes it easy to run more experiments, the surface area for competitive intelligence exposure grows. Growth engineers should build experiment naming conventions and API obfuscation into their platform standards from the start.

7. Growth Marketing Lens

7.1 The TAM Expansion: Marketing to "Aspiring Builders"

Boris's printing press analogy is not just a philosophical framework; it is a market sizing argument. If AI makes software creation accessible to non-engineers, the total addressable market for coding tools, development platforms, and software creation products expands by an order of magnitude. The printing press didn't just serve existing scribes more efficiently; it created an entirely new audience of readers. AI coding tools are doing the same: creating an audience of builders who never would have built anything under the old cost structure.

Elena Verna's work at Lovable demonstrates what this looks like in practice. Lovable hit $200M ARR in one year with 100 employees, and their most powerful growth strategy was giving the product away for free, because the product itself generates word-of-mouth among people who couldn't previously build software. The product is the marketing. This is PLG in its purest form: freemium conversion in this category draws from a TAM that didn't exist two years ago.

For growth marketers at AI-native companies, this demands a rethinking of ICP definition. The traditional developer tool ICP (professional software engineers at companies with 50+ developers) is now just one segment. The new segments include: solo founders building MVPs without technical co-founders, product managers prototyping features without filing engineering tickets, designers shipping their own implementations, and knowledge workers automating their own workflows. Each of these segments has different acquisition channels, different messaging, and different activation metrics. The growth marketer's job is to identify which segment has the fastest path to revenue and build the acquisition engine around it.

Notion's growth model offers a template. They achieved 95% organic traffic through community-led growth, with 280,000+ subreddit members and thousands of user-made templates driving onboarding for new users. The parallel for AI coding tools: user-generated prompts, shared agent configurations, and community-built templates could serve the same function, turning every successful build into a new user acquisition asset.

7.2 Thought Leadership as the Highest-Leverage Acquisition Channel

Boris's appearance on Lenny's Podcast is itself a growth marketing case study worth dissecting. The episode generated 423,000+ views and triggered coverage across VentureBeat, Fortune, TechCrunch, and tech Twitter. For growth marketers at AI companies, this illustrates a channel strategy that is particularly effective in the current environment.

The reason executive thought leadership works so well for AI products is rooted in Brian Balfour's Four Fits framework. The product-channel fit for AI coding tools requires channels that can convey both capability and trust simultaneously. Paid search can drive trial signups, but it cannot convey the nuance of "coding is largely solved, with important exceptions." A 75-minute podcast conversation can. The channel matches the product's complexity.

This connects to the CAC crisis hitting B2B SaaS. Average B2B paid search CAC is $802 in 2025, and CAC has risen 40 to 60% since 2023. Referrals remain the most cost-efficient channel at $141 to $200 per customer. Thought leadership operates in the referral range: the cost is primarily executive time, and the distribution is organic. When Boris says something provocative ("I haven't written a single line of code since November"), it spreads virally through tech communities without paid amplification.

Emily Kramer's GACC framework (Goals, Audience, Creative, Channel) clarifies why this works at scale. The goal is awareness and consideration among a technical audience. The audience is growth-stage engineering leaders evaluating AI tools. The creative is the executive's authentic perspective (not marketing copy). The channel is high-reach, high-trust media (Lenny's Podcast reaches hundreds of thousands of practitioners). Every element is aligned, which is why the episode drove measurable adoption.

For growth marketers building their own thought leadership pipeline, the operational insight is that consistency matters more than any single hit. Amanda Natividad's concept of "zero-click content" (content that delivers value without requiring a click) applies here: the value is in the reach and trust built over time, not in driving clicks to a landing page from any individual appearance.

7.3 Channel Strategy in a Zero-Click, AI-Mediated World

Boris's claim that AI is changing how software gets built has a parallel in how software gets

discovered and adopted. The traditional growth marketing funnel (paid ad to landing page to trial signup to activation) is under structural pressure from multiple directions.

SEO is now AEO (Answer Engine Optimization). 58.5% of US Google searches end without a click. AI Overviews reduce organic CTR by 61% (from 1.76% to 0.61%). Even more striking, 93% of AI Mode searches end without a click versus 43% for standard AI Overviews. For growth marketers, this means the SEO playbook that worked in 2020 is increasingly broken. The new game is getting cited by AI systems rather than ranking in traditional results. Being cited in AI Overviews increases organic CTR by 35% compared to not being cited, and AI-referred visitors convert 23x higher than organic search visitors (lower volume, far higher quality).

The practical implication for AI coding tool marketing: optimize content for AI citation, not just search ranking. This means structured content that AI can extract and reference, depth over breadth (44.2% of LLM citations come from the first 30% of text), and genuine E-E-A-T signals. Pure AI-generated content performs terribly here, with only 3% of AI-generated pages remaining in the top 100 after three months.

PLG mechanics matter more than ever. 91% of B2B SaaS companies are increasing PLG investment in 2025. Product-qualified leads (PQLs) convert at 25 to 30% versus 5 to 10% for traditional MQLs. PLG companies acquire customers at roughly one-tenth the cost of sales-led companies ($100 to $500 CAC versus $5,000 to $50,000). Boris's description of Claude Code going from weekend prototype to 4% of public GitHub commits is a PLG story: the product's adoption was driven by the product itself, not by a sales team or advertising budget.

For growth marketers at AI companies, the PLG playbook is: make the product free or very cheap to start, optimize time-to-value so users experience the "aha moment" fast (Boris's "latent demand" framing suggests the aha moment is already understood by users; they just need the tool to deliver it), and build viral mechanics into the product (shared agent configurations, collaborative coding sessions, public project templates). Reverse trials (giving full access upfront, then gating after a time limit) are particularly effective because they let users experience the "coding is solved" moment before hitting a paywall.

Community-led growth as a compounding channel. Figma's most powerful growth engine was templates (discovered in 2018): users building UI kits, icon sets, and design systems and sharing publicly. Notion achieved 95% organic traffic through community-driven growth. For AI coding tools, the equivalent is user-generated prompts, agent configurations, and

workflow templates that both demonstrate the product's capability and directly onboard new users. Each community contribution is a new acquisition surface, creating the growth loop (not funnel) that Brian Balfour describes: output (user-created content) feeds back into input (new user discovery and activation).

7.4 Pricing and Monetization in the "Unlimited Tokens" Era

Boris's recommendation that companies give engineers unlimited AI tokens connects to the biggest pricing upheaval in SaaS history. Kyle Poyar's research shows that among the top 500 SaaS and AI companies, there were 1,800+ pricing changes in 2025 (3.6 per company on average). Credit-based pricing models exploded, up 126% year over year. Hybrid pricing (subscription plus usage) is emerging as the standard.

For growth marketers, Boris's "unlimited tokens" philosophy creates a specific positioning challenge: how do you monetize a product when the founder's advice is to give the core resource away? The answer lies in how Anthropic and similar companies are navigating this: free or low-cost access for individual usage (driving PLG adoption and community growth), tiered pricing based on usage volume and team features (capturing value from organizations), and enterprise pricing based on seats, governance, and security requirements.

This maps onto the broader trend Poyar identifies: seats and flat-rate pricing are under threat as AI disconnects value from login counts. If one engineer with unlimited tokens is as productive as five without them (Boris's 200% productivity claim suggests this is conservative), per-seat pricing undervalues the product. Usage-based or outcome-based pricing better captures the value AI tools deliver. Growth marketers need to design pricing experiments that test these models, recognizing that the optimal pricing structure is still being discovered across the industry.

The freemium conversion benchmark (5% of freemium signups converting to paid, per OpenView) sets a baseline, but AI coding tools may outperform this because the value proposition is so immediately demonstrable. When a user sees AI write working code in their first session, the aha moment is visceral. Growth marketers should measure time-to-first-successful-build as a leading indicator of conversion and optimize the onboarding experience to minimize this metric.

7.5 Retention Through Feature Velocity and the Content Loop

Boris's claim that AI makes engineering faster is not just an engineering story; it is a retention marketing story. Spotify's CEO stated that their best developers haven't written a line of code since December 2025, and the result was shipping 50+ new features throughout the year. The retention loop is clear: faster engineering velocity leads to more features, more features deliver more user value, more value drives higher retention, and higher retention improves LTV.

For growth marketers, this means feature velocity should be part of the marketing narrative, not just the engineering narrative. Changelogs, release notes, and product update communications become retention marketing assets. The growth marketer's job is to connect each new feature to the user's job-to-be-done and communicate it through lifecycle email sequences, in-app announcements, and community updates.

The data supports the investment in lifecycle marketing infrastructure. AI-powered email personalization increases revenue by up to 41% and lifts CTR by over 13%. Omnisend reports $79 revenue for every $1 spent on combined AI-powered automation and real-time personalization across email and SMS. For AI coding tools with high feature velocity, the lifecycle marketing engine becomes a compounding retention loop: every new feature is a reason to re-engage churning or dormant users.

Elena Verna's anti-pattern #7 (relying on one-off emails for growth) is relevant here. The temptation with high feature velocity is to send blast announcements for each release. The better approach is building automated lifecycle sequences triggered by user behavior: if a user hasn't tried the multi-agent workflow Boris describes, trigger a sequence that demonstrates it. If a user's experiment velocity has dropped, surface the latest AI-assisted experiment features. These behavioral triggers convert feature velocity into measurable retention lift.

7.6 Attribution and Dark Social in the AI Tool Market

Boris's description of Claude Code growing from a weekend prototype to 4% of GitHub commits raises an important attribution question: how much of that growth came from measurable channels versus dark social (private shares via DMs, Slack groups, WhatsApp, and text messages)?

For growth marketers at AI tools companies, attribution is structurally harder than in traditional SaaS. Third-party cookies are already blocked in Safari and Firefox. Cross-site tracking gaps mean browsers block tracking pixels and cannot connect sessions to original ad clicks. Retargeting pools have shrunk dramatically. Meanwhile, the most powerful acquisition channel for developer tools (one engineer telling another in a Slack channel or Twitter DM) is almost entirely unmeasurable.

Elena Verna's framework is useful here: prioritize owned and earned channels (virality, word of mouth, UGC) over renting access to others' audiences via paid channels. First-party data is increasingly critical: 71% of publishers recognize it as key (up from 64% in 2024), and 85% expect its role to increase in 2026.

The practical move for growth marketers is to build attribution infrastructure that accounts for dark social. Self-reported attribution ("how did you hear about us?") on signup forms, unique referral codes that travel through dark channels, and correlation analysis between thought leadership appearances (like Boris's Lenny Podcast episode) and signup spikes all help paint a more complete picture. The growth marketer who can connect Boris's 423,000-view podcast episode to a measurable signup cohort, even imperfectly, provides the kind of data-informed insight that justifies continued investment in organic channels.

7.7 The ABM Angle: Selling AI Coding Tools to Enterprises

Boris describes Anthropic's policy of giving every engineer unlimited AI tokens. For growth marketers selling AI coding tools into enterprises, this policy is itself the value proposition: prove that unlimited AI tokens generate measurable ROI (Boris's 200% productivity claim), and the sale closes itself.

ABM data supports this approach. Companies implementing ABM see 16% more opportunities on average, 60% higher win rates when aligned with account-based advertising, and 171% ACV lift after implementation. For AI coding tools, the ABM play is to identify engineering-led organizations where the CTO or VP of Engineering is already experimenting with AI coding tools (intent signals from tool evaluations, blog posts about AI adoption, GitHub activity with AI-generated code patterns), then run targeted campaigns that translate Boris's claims into enterprise-specific ROI language.

Teams acting on intent spikes within 24 hours see a 29% lift in opportunity creation.

AI-assisted account scoring increases opportunity creation by 38% when combined with human QA. For growth marketers, this means building a pipeline that monitors public signals of AI tool adoption (GitHub commit patterns, engineering blog posts, job postings mentioning AI coding tools) and triggers ABM sequences that position your product as the next step in their AI adoption journey.

---

8. Contrarian / Non-Obvious Takes

8.1 The Scribe Was Excited

The most counterintuitive moment in the conversation is the historical detail about scribes and the printing press. The standard narrative is that technology displaces workers who resist change. But the historical record shows that skilled scribes were actually excited about the press because it freed them from copying. Boris identifies with this: he says he's never enjoyed coding more than he does now, precisely because the tedious parts are gone. The contrarian take for practitioners: the best engineers will embrace AI coding because it eliminates the parts of the job they never liked.

8.2 Underfunding as a Feature

Conventional wisdom says important projects should be generously resourced. Boris argues the opposite for AI products. Small teams with limited headcount but unlimited AI tokens produce better products because the constraints force them to use their own tools intensively, which improves the product through better dogfooding. This is a genuine inversion of standard scaling logic.

8.3 The Model Matters More Than the Tool

While the market obsesses over editor features (Cursor's UI, Windsurf's experience, Copilot's integrations), Boris makes the case that the underlying model quality is the decisive factor. He uses Opus despite it being slower and more expensive than Sonnet because it requires less steering and produces better results per interaction. The contrarian take: the "best coding tool" is whichever one has the best model, regardless of UX.

8.4 Anxiety Is the Correct Response

Most tech leaders project calm confidence. Boris openly acknowledges that working at the frontier is anxiety-inducing, and he suggests that the absence of anxiety means you're not paying attention. This is a genuinely non-obvious position in an industry that rewards optimism and certainty.

---

9. Quotable Moments

1. **"100% of my code is written by Claude Code"** : the opening statement that sets the scale of the shift. Not 80%, not most. All of it.
2. **"Coding is largely solved"** : the thesis in four words. Provocative, but carefully qualified throughout the conversation.
3. **"Give engineers unlimited tokens"** : a specific, implementable recommendation that challenges how organizations budget for AI tools.
4. **"The scribe was actually excited"** : the historical detail that reframes the displacement narrative entirely.
5. **"In a year or two, it's not going to matter"** : on whether people should learn to code. The starkest prediction in the conversation.

---

10. What to Revisit

- **(00:00 to 03:45) The opening montage** : Boris's workflow described in his own words. Watch this to internalize what "100% AI-written code" looks like in practice.
- **(03:45 to 05:35) Why he left for Cursor and came back** : the competitive insight about model quality versus editor features.
- **(08:41 to 13:29) The origin story of Claude Code** : how a weekend hack became 4% of GitHub commits. Essential for understanding product-market fit and latent demand.
- **(16:17 to 17:32) Boris's current workflow** : the specific details of running 5+ agents simultaneously. Model your own setup after this.
- **(24:02 to 26:48) Principles for the Claude Code team** : the three principles Boris gives every new team member.
- **(26:48 to 27:55) Why unlimited tokens matter** : the business case for giving engineers unlimited AI compute. Share this section with your VP of Engineering.
- **(32:15 to 36:01) The printing press analogy** : the best-articulated framework in the conversation. Revisit when explaining AI's long-term impact to non-technical stakeholders.
- **(46:32 to 51:53) Latent demand and Cowork** : how Anthropic identified and validated unmet demand. Essential product strategy material.
- **(54:04 to 59:35) AI safety at Anthropic** : the three layers framework. Useful context for anyone deploying AI tools in production.
- **(1:03:21 to 1:08:38) Advice for AI products and Claude Code tips** : concentrated tactical advice. Revisit before starting any AI-assisted project.

---

11. Sources & Further Reading

1. **Gergely Orosz, "Building Claude Code with Boris Cherny" (The Pragmatic Engineer)**

https://newsletter.pragmaticengineer.com/p/building-claude-code-with-boris-cherny Deep technical dive into Claude Code's architecture and Boris's engineering philosophy.

2. **Lenny Rachitsky's episode page and transcript**

https://www.lennysnewsletter.com/p/head-of-claude-code-what-happens Full episode transcript and Lenny's own takeaways.

3. **TechCrunch: Spotify's best developers haven't written code since December**

https://techcrunch.com/2026/02/12/spotify-says-its-best-developers-havent-written-a-line-of-code-since-december Enterprise case study validating Boris's claims about AI-written code at scale.

4. **Fortune: Claude Code gives Anthropic its viral moment**

https://fortune.com/2026/01/24/anthropic-boris-cherny-claude-code-non-coders-software-engineers/ Business context on Claude Code's market impact and growth trajectory.

5. **METR: Measuring the Impact of Early-2025 AI on Developer Productivity**

https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/ Rigorous RCT showing AI tools made experienced developers 19% slower on certain tasks. Essential counterbalance to productivity optimism.

6. **Elena Verna on Lenny's Podcast: The new AI growth playbook for 2026**

https://www.lennysnewsletter.com/p/the-new-ai-growth-playbook-for-2026-elena-verna Elena's framework for how growth loops, distribution, and PLG are changing in the AI era.

7. **2025 DORA Report on AI and Software Engineering Performance (InfoQ)**

https://www.infoq.com/news/2026/03/ai-dora-report/ Industry benchmark data on how AI adoption correlates with engineering delivery metrics.

8. **Waydev: 8 Game-Changing Insights from Anthropic's Boris Cherny**

https://waydev.co/8-game-changing-insights-from-anthropic-claudecode-boris-cherny/ Structured summary of Boris's key claims with supporting data.

9. **Boris Cherny's personal site and Twitter (@bcherny)**

https://borischerny.com / https://x.com/bcherny His viral thread on his Claude Code setup and ongoing commentary.

10. **Faros AI: The AI Productivity Paradox Research Report**

https://www.faros.ai/blog/ai-software-engineering Data on the gap between individual AI

productivity gains and organizational delivery velocity.